

Advanced Async Patterns

JavaScript Advanced — Senior Developer Level

এই section-এর 9টি topic:

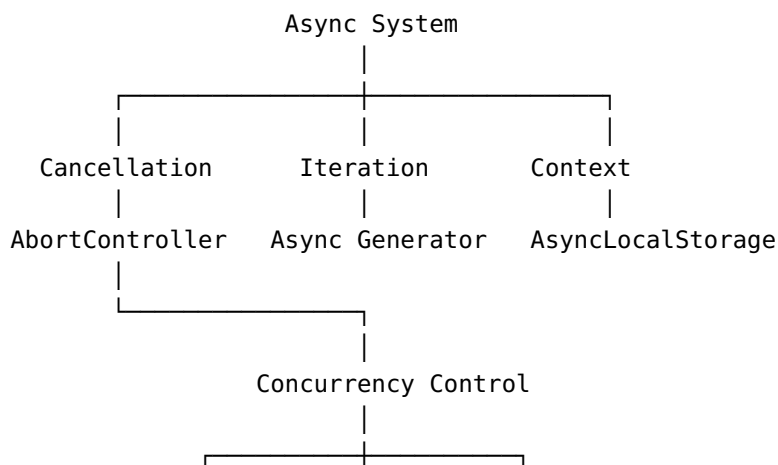
1. `AbortController` & `AbortSignal`
2. Async Generator Functions
3. `for await...of`
4. Node.js `AsyncLocalStorage`
5. Concurrency Limiter
6. Retry with Exponential Backoff
7. Circuit Breaker
8. Promise Queue
9. Async Resource Management — `using`

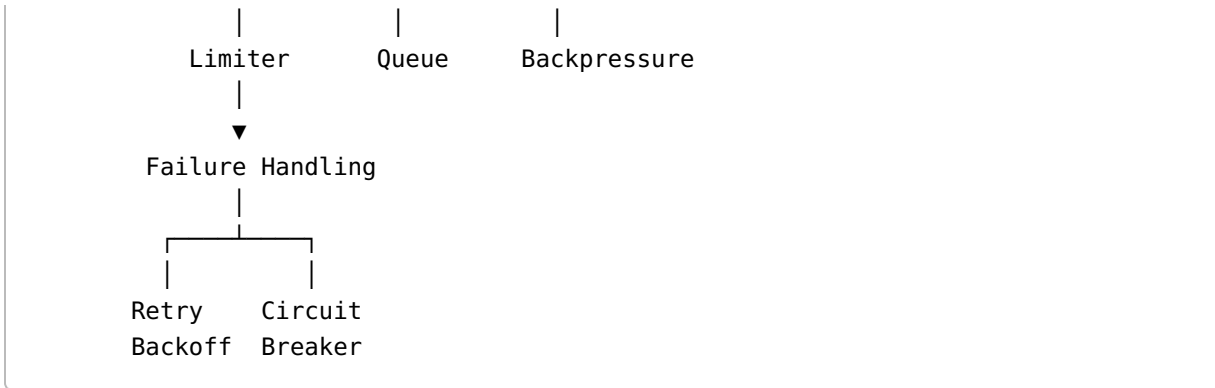
1. Advanced Async-এর বড় picture

Basic async JavaScript:

```
Promise
  ↓
async / await
  ↓
Concurrent I/O
```

Advanced async architecture:





Production system-এ শুধু:

```
await fetch(...)
```

জানলেই যথেষ্ট নয়।

তোমাকে জানতে হবে:

- কখন request cancel করবে?
- কতগুলো কাজ একসাথে চলবে?
- failure হলে retry করবে?
- কতবার retry করবে?
- dependency down হলে কী করবে?
- request-এর context async chain-এর মধ্যে কীভাবে রাখবে?
- stream-like data কীভাবে consume করবে?
- resource automatically cleanup করবে কীভাবে?

2. AbortController & AbortSignal

2.1 Problem

ধরো:

```
const response = await fetch("/api/users");
```

Request শুরু হলো।

কিন্তু user page থেকে চলে গেল।

তখন request-এর আর দরকার নেই।

যদি cancellation mechanism না থাকে:

```
User leaves page
  ↓
Request still running
  ↓
Network/resource wasted
  ↓
Response eventually arrives
```

এখানে cancellation দরকার।

3. AbortController

`AbortController` cancellation signal তৈরি করার জন্য ব্যবহৃত হয়।

```
const controller = new AbortController();

const signal = controller.signal;
```

তারপর:

```
fetch("/api/users", {
  signal
});
```

যখন cancel করতে চাও:

```
controller.abort();
```

Flow:

```
AbortController
  |
  ▼
AbortSignal
  |
  ▼
fetch(...)
  |
```

```
|
controller.abort()
|
▼
Request cancelled
```

4. AbortSignal কী?

AbortSignal হলো cancellation-এর state/information বহনকারী object ।

```
const controller = new AbortController();

console.log(controller.signal.aborted);
// false

controller.abort();

console.log(controller.signal.aborted);
// true
```

অর্থাৎ:

```
AbortController
├── signal
│   ├── aborted
│   └── abort event
```

5. Fetch Cancellation

```
const controller = new AbortController();

async function loadUsers() {
  try {
    const response = await fetch("/api/users", {
      signal: controller.signal
    });
  }
}
```

```

    return await response.json();
  } catch (error) {
    console.log(error);
  }
}

loadUsers();

// later
controller.abort();

```

Cancellation হলে fetch সাধারণত rejected Promise দেয়।

তাই production code-এ cancellation আলাদাভাবে handle করা ভালো।

```

try {
  await fetch(url, {
    signal: controller.signal
  });
} catch (error) {
  if (error.name === "AbortError") {
    console.log("Request cancelled");
    return;
  }

  throw error;
}

```

6. AbortController শুধু fetch-এর জন্য নয়

এটা খুব গুরুত্বপূর্ণ।

AbortSignal একটি generic cancellation mechanism।

নিজের async function-এও ব্যবহার করতে পারো:

```

async function doWork(signal) {
  if (signal.aborted) {
    throw new Error("Operation aborted");
  }
}

```

```
// work...  
}
```

আর abort event:

```
signal.addEventListener("abort", () => {  
  console.log("Cancelled");  
});
```

7. AbortSignal.throwIfAborted()

Modern environments-এ:

```
signal.throwIfAborted();
```

ব্যবহার করা যায়।

```
async function processItems(items, signal) {  
  for (const item of items) {  
    signal.throwIfAborted();  
  
    await process(item);  
  }  
}
```

এতে cancellation state check concise হয়।

8. AbortSignal.timeout()

Timeout-এর জন্য modern API:

```
const signal = AbortSignal.timeout(5000);  
  
await fetch("/api/users", {  
  signal  
});
```

অর্থ:

```
Request
  ↓
5 seconds
  ↓
not completed
  ↓
abort
```

এটি manually timer + controller setup করার প্রয়োজন কমায়।

9. AbortSignal.any()

একাধিক cancellation condition combine করা যায়।

```
const controller = new AbortController();

const signal = AbortSignal.any([
  controller.signal,
  AbortSignal.timeout(5000)
]);

fetch("/api/users", {
  signal
});
```

এখানে:

```
User cancellation ┌───┐
                    │   │
                    └───┘→ Abort
5 second timeout ┌───┐
```

যেকোনো একটি abort করলে combined signal aborted হবে।

10. Cancellation $\neq$ Promise rejection only

খুব important distinction:

```
AbortController
  ↓
asks operation to cancel
```

এটা CPU magically undo করে না।

যদি তুমি নিজের function লিখো:

```
async function work(signal) {
  while (...) {
    // huge synchronous calculation
  }
}
```

তাহলে মাঝখানে cancellation check না করলে:

```
controller.abort();
```

দিলেও synchronous loop নিজে থেকে থামবে না।

তাই cooperative cancellation দরকার।

11. Async Generator Functions

Syntax:

```
async function* numbers() {
  yield 1;
  yield 2;
  yield 3;
}
```

এখানে:

```
async function*
  ↓
Async Generator
```

এটি একসাথে:

- async
- generator
- iterable

concept combine করে।

12. Normal Generator বনাম Async Generator

Normal generator:

```
function* numbers() {  
  yield 1;  
  yield 2;  
}
```

Async generator:

```
async function* numbers() {  
  yield 1;  
  yield 2;  
}
```

Normal:

Iterator

Async:

AsyncIterator

13. Async Generator কী return করে?

```
const iterator = numbers();  
  
console.log(iterator.next());
```

Async generator-এর `next()` Promise return করে।

```
const result = await iterator.next();  
  
console.log(result);
```

Result:

```
{  
  value: 1,  
  done: false  
}
```

Conceptually:

```
iterator.next()  
  ↓  
Promise  
  ↓  
{ value, done }
```

14. Async Generator-এর মূল সুবিধা

ধরো server থেকে data chunks আসছে:

```
chunk 1  
  ↓  
wait  
  ↓  
chunk 2  
  ↓  
wait
```

↓
chunk 3

সব একসাথে memory-তে load করার পরিবর্তে:

```
async function* fetchPages() {  
  yield await fetchPage(1);  
  yield await fetchPage(2);  
  yield await fetchPage(3);  
}
```

Consumer একেকটি result একেক সময় পেতে পারে।

15. Async Generator Example

```
async function* fetchUsers() {  
  for (let page = 1; page <= 3; page++) {  
    const response = await fetch(`/users?page=${page}`);  
    const data = await response.json();  
  
    yield data;  
  }  
}
```

Consumer:

```
for await (const users of fetchUsers()) {  
  console.log(users);  
}
```

Flow:

```
fetch page 1  
  ↓  
yield  
  ↓  
consumer processes  
  ↓  
fetch page 2  
  ↓  
yield
```

↓
consumer processes

এটি lazy async data production-এর জন্য খুব useful।

16. Async Generator-এর `.return()` এবং `.throw()`

Async generators iterator protocol-এর methods support করে:

```
next()  
return()  
throw()
```

যেমন:

```
const iterator = fetchUsers();  
  
await iterator.return();
```

এটি generator-এর execution বন্ধ করার জন্য ব্যবহৃত হতে পারে।

এটি cleanup logic-এর সঙ্গে গুরুত্বপূর্ণ।

17. for await...of

Async iterable consume করার syntax:

```
for await (const item of iterable) {  
  console.log(item);  
}
```

Example:

```
async function* generate() {  
  yield 1;  
  await delay(1000);  
  yield 2;  
  await delay(1000);  
}
```

```
    yield 3;
  }

  for await (const value of generate()) {
    console.log(value);
  }
```

Output সময়ের সঙ্গে:

```
1
...1 sec...
2
...1 sec...
3
```

18. `for await...of` শুধু Async Generator-এর জন্য নয়

এটি async iterable consume করতে পারে।

Conceptually:

```
Async Iterable
  ↓
[Symbol.asyncIterator]()
  ↓
Async Iterator
  ↓
next()
  ↓
Promise
```

তাই async generator শুধু একটি common way।

19. Async Iterator Protocol

Async iterable-এর গুরুত্বপূর্ণ symbol:

```
Symbol.asyncIterator
```

একটি object async iterable হতে পারে যদি এটি:

```
obj[Symbol.asyncIterator]
```

provide করে।

Example:

```
const iterable = {  
  async *[Symbol.asyncIterator]() {  
    yield 1;  
    yield 2;  
    yield 3;  
  }  
};
```

তারপর:

```
for await (const value of iterable) {  
  console.log(value);  
}
```

20. `for await...of` বনাম `for...of`

Normal:

```
for (const item of items) {  
  console.log(item);  
}
```

Async:

```
for await (const item of items) {  
  console.log(item);  
}
```

Difference:

for...of	for await...of
Sync iterator	Async iterator
synchronous values	Promise-like async values
immediate iteration	awaits each iteration

21. AsyncLocalStorage

এটি Node.js-এর খুব important advanced feature ।

Problem:

একটি HTTP request handle করার সময় তুমি context রাখতে চাও:

```
requestId
userId
traceId
transaction context
```

কিন্তু async operations-এর মধ্যে এই context কীভাবে preserve করবে?

22. Bad Approach — Global Variable

```
let currentRequestId;

app.get("/users", async (req, res) => {
  currentRequestId = req.id;

  await database.query();

  console.log(currentRequestId);
});
```

এটি dangerous ।

কারণ concurrent requests:

```
Request A
requestId = A

Request B
requestId = B

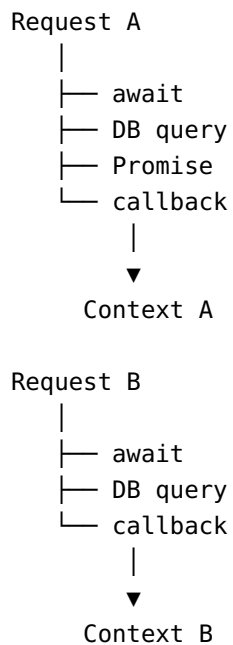
Request A resumes
requestId = B ❌
```

Race/context contamination হতে পারে।

23. AsyncLocalStorage কী solve করে?

Node.js `AsyncLocalStorage` asynchronous execution-এর সঙ্গে context associate করতে দেয়।

Conceptually:



দুটো context আলাদা থাকে।

24. Basic AsyncLocalStorage Example

```
import { AsyncLocalStorage } from "node:async_hooks";

const storage = new AsyncLocalStorage();
```

Request-এর সময়:

```
app.use((req, res, next) => {
  const context = {
    requestId: crypto.randomUUID()
  };

  storage.run(context, next);
});
```

তারপর অন্য function-এ:

```
function log(message) {
  const context = storage.getStore();

  console.log(
    context?.requestId,
    message
  );
}
```

এখন `log()` -এর কাছে request object manually pass করতে হচ্ছে না।

25. কেন AsyncLocalStorage Useful?

বিশেষ করে:

Request tracing

```
requestId
```

Logging

```
[request-123] Database query started  
[request-123] Database query finished
```

Distributed tracing

```
traceId  
spanId
```

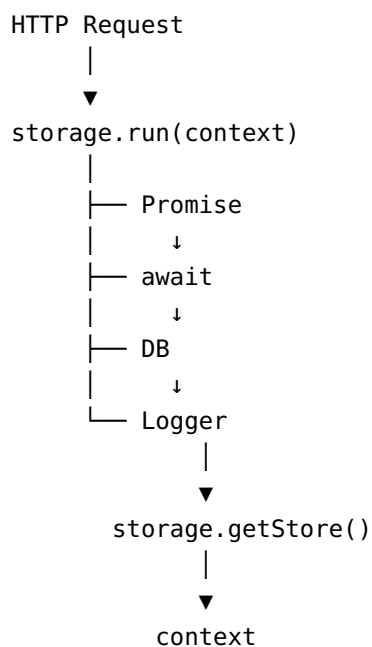
Request-scoped metadata

```
tenantId  
userId  
locale
```

Transaction-related context

কিছু application architecture-এ request-scoped transaction/context রাখতেও ব্যবহার করা যায়।

26. AsyncLocalStorage Mental Model



এখানে মূল বিষয়:

context async execution chain-এর সঙ্গে propagate করে।

27. Concurrency কী?

ধরো 1000টা কাজ আছে:

```
const results = await Promise.all(  
  items.map(processItem)  
);
```

এতে potentially অনেক operation একসাথে start হতে পারে।

যদি:

1000 requests

একসাথে database/API-তে পাঠাও:

```
API  
↑ ↑ ↑ ↑ ↑ ↑ ↑ ↑  
1000 concurrent requests
```

সমস্যা হতে পারে:

- rate limit
- memory usage
- connection pool exhaustion
- server overload
- network saturation

তাই **concurrency limiter** দরকার।

28. Concurrency Limiter

ধরো:

N = 100 tasks
K = 5 concurrent

মানে:

মোট 100 কাজ থাকবে, কিন্তু একই সময়ে সর্বোচ্চ 5টি চলবে।

Task 1 ┘
Task 2 ┘
Task 3 ┘ → running
Task 4 ┘
Task 5 ┘

Task 6+
waiting

একটি শেষ হলে:

Task 1 done
↓
Task 6 starts

29. Simple Concurrency Limiter

```
async function mapWithConcurrency(items, limit, worker) {  
  const results = new Array(items.length);  
  let nextIndex = 0;  
  
  async function run() {  
    while (true) {  
      const index = nextIndex++;  
  
      if (index >= items.length) {  
        return;  
      }  
  
      results[index] = await worker(items[index], index);  
    }  
  }  
  
  const workers = Array.from(  

```

```

    { length: Math.min(limit, items.length) },
    () => run()
  );

  await Promise.all(workers);

  return results;
}

```

ব্যবহার:

```

const results = await mapWithConcurrency(
  users,
  5,
  user => fetchUserData(user)
);

```

এখানে:

```

users = 100
limit = 5

```

একসাথে সর্বোচ্চ 5 worker loop active থাকবে।

30. Concurrency বনাম Parallelism

এগুলো গুলিয়ে ফেলো না।

Concurrency

একাধিক কাজ progress করতে পারে।

```

A —wait→
B →
C —wait→

```

Parallelism

একই সময়ে multiple CPU execution হতে পারে।

```
CPU 1 → A
CPU 2 → B
CPU 3 → C
```

JavaScript async I/O-তে concurrency অনেক common ।

Worker Threads ব্যবহার করলে actual CPU parallelism সম্ভব হতে পারে ।

31. Retry Pattern

Network failure:

```
await fetch(url);
```

fail করতে পারে ।

Temporary failure হলে আবার চেষ্টা করা যেতে পারে ।

```
Attempt 1
  ↓ fail
Attempt 2
  ↓ fail
Attempt 3
  ↓ success
```

এটাই retry pattern ।

32. কেন Immediately Retry করা খারাপ?

ধরো server overloaded:

```
Client
  ↓
Request
  ↓
Server overloaded
  ↓
Retry immediately
  ↓
```

```
Server overloaded
```

```
↓
```

```
Retry
```

```
↓
```

```
...
```

এতে load আরও বাড়বে।

এটাকে **retry storm** বলা যেতে পারে।

তাই delay দরকার।

33. Exponential Backoff

প্রতিবার retry delay বাড়ানো:

```
Attempt 1 → 100ms  
Attempt 2 → 200ms  
Attempt 3 → 400ms  
Attempt 4 → 800ms  
Attempt 5 → 1600ms
```

Formula:

```
delay = base × 2^attempt
```

Example:

```
const delay = 100 * 2 ** attempt;
```

34. Jitter

সব clients একই সময়ে retry করলে সমস্যা হতে পারে।

ধরো 10,000 client:

```
100ms  
200ms  
400ms
```

সবাই একই সময়ে request পাঠাচ্ছে।

Jitter randomizes the delay:

```
const jitter = Math.random() * 100;  
const delay = baseDelay + jitter;
```

আরও common approach:

```
exponential backoff + jitter
```

Production distributed systems-এ এটি খুব useful।

35. Retry Example

```
function sleep(ms) {  
  return new Promise(resolve => {  
    setTimeout(resolve, ms);  
  });  
}  
  
async function retry(fn, options = {}) {  
  const {  
    retries = 3,  
    baseDelay = 200  
  } = options;  
  
  let attempt = 0;  
  
  while (true) {  
    try {  
      return await fn();  
    } catch (error) {  
      if (attempt >= retries) {  
        throw error;  
      }  
    }  
  }  
}
```

```
const delay =
  baseDelay * 2 ** attempt;

await sleep(delay);

attempt++;
}
}
}
```

ব্যবহার:

```
const data = await retry(
  () => fetch("/api/data"),
  {
    retries: 4,
    baseDelay: 200
  }
);
```

36. কখন **Retry** করা উচিত নয়?

সব error retryable না।

যেমন:

```
400 Bad Request
401 Unauthorized
403 Forbidden
404 Not Found
```

সাধারণত blind retry অর্থপূর্ণ নয়।

Temporary infrastructure failures:

```
network failure
timeout
429 rate limit
5xx
```

কিছু ক্ষেত্রে retryable হতে পারে।

কিন্তু exact policy API/service-এর semantics-এর ওপর নির্ভর করবে।

37. Idempotency এবং Retry

এটি Senior-level খুব important।

ধরো:

```
POST /payment
```

Request server-এ পৌঁছেছে।

কিন্তু response পথে network failure হলো।

Client ভাবে:

```
Request failed  
↓  
Retry
```

দ্বিতীয়বার payment charge হতে পারে।

তাই retry-এর সঙ্গে **idempotency** চিন্তা করতে হবে।

উদাহরণ:

```
Idempotency-Key: abc123
```

Server একই logical operation duplicate হলে detect করতে পারে।

38. Circuit Breaker Pattern

Retry সবসময় solution না।

ধরো dependency permanently/down temporarily unavailable:

```
Your API  
↓
```

Payment Service



DOWN

তুমি প্রতিটি request-এ retry করলে:

1000 requests



5000 retries

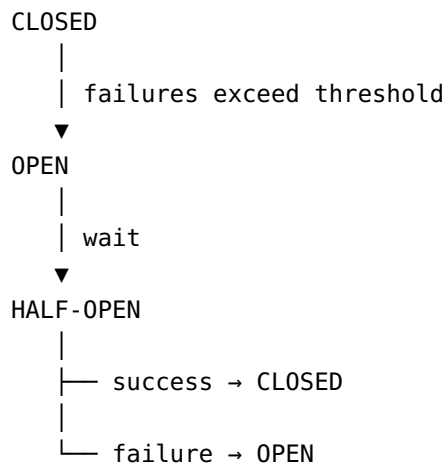


more load

এখানে circuit breaker দরকার।

39. Circuit Breaker States

সাধারণত 3টি state:



40. CLOSED

Normal operation:

Request



Dependency

↓
Success

Failures count করা হয়।

failureCount++

Threshold cross করলে:

CLOSED → OPEN

41. OPEN

Circuit open হলে dependency-তে request পাঠানো বন্ধ।

Request
↓
Circuit OPEN
↓
Fail fast

এতে dependency-এর ওপর additional load যায় না।

42. HALF-OPEN

কিছু সময় পরে test request:

OPEN
↓
cooldown
↓
HALF-OPEN
↓
test request

যদি success:

HALF-OPEN → CLOSED

যদি failure:

HALF-OPEN → OPEN

43. Simple Circuit Breaker Model

```
class CircuitBreaker {
  constructor(fn, options = {}) {
    this.fn = fn;
    this.failureThreshold =
      options.failureThreshold ?? 3;

    this.resetTimeout =
      options.resetTimeout ?? 5000;

    this.failures = 0;
    this.state = "CLOSED";
    this.openedAt = 0;
  }

  async execute(...args) {
    if (this.state === "OPEN") {
      if (
        Date.now() - this.openedAt <
        this.resetTimeout
      ) {
        throw new Error("Circuit is open");
      }

      this.state = "HALF_OPEN";
    }

    try {
      const result = await this.fn(...args);

      this.failures = 0;
      this.state = "CLOSED";

      return result;
    } catch (error) {
```

```

        this.failures++;

        if (
            this.failures >=
            this.failureThreshold
        ) {
            this.state = "OPEN";
            this.openedAt = Date.now();
        }

        throw error;
    }
}
}

```

এটি conceptual implementation; production circuit breakers সাধারণত metrics, timeout, half-open probe limits, error classification এবং observability আরও carefully handle করে।

44. Retry বনাম Circuit Breaker

দুটো একই problem solve করে না।

Retry

Temporary failure?
↓
Try again

Circuit Breaker

Dependency repeatedly failing?
↓
Stop calling it temporarily

একসাথে ব্যবহার করা যায়:

Request
↓
Circuit check
↓
Allowed?
↓

```
Retry with backoff
↓
Still failing
↓
Record failure
↓
Open circuit
```

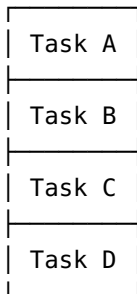
45. Promise Queue

Promise Queue হলো async কাজগুলোকে queue করে controlledভাবে execute করার pattern।

ধরো:

```
Task A
Task B
Task C
Task D
```

Queue:



Worker একেকটা করে process করতে পারে।

46. Sequential Promise Queue

Simple queue:

```
class PromiseQueue {
  constructor() {
```

```

    this.queue = [];
    this.running = false;
  }

  add(task) {
    return new Promise((resolve, reject) => {
      this.queue.push({
        task,
        resolve,
        reject
      });

      this.run();
    });
  }

  async run() {
    if (this.running) return;

    this.running = true;

    while (this.queue.length > 0) {
      const { task, resolve, reject } =
        this.queue.shift();

      try {
        resolve(await task());
      } catch (error) {
        reject(error);
      }
    }

    this.running = false;
  }
}

```

ব্যবহার:

```

const queue = new PromiseQueue();

queue.add(() => fetch("/a"));
queue.add(() => fetch("/b"));
queue.add(() => fetch("/c"));

```

Execution:

A
↓
B
↓
C

47. Promise Queue বনাম Concurrency Limiter

এখানে difference খুব important।

Promise Queue — concurrency 1

A
↓
B
↓
C

Concurrency limiter — concurrency K

A ┌
B ┌
C ┌

D waits

A finishes

↓

D starts

অর্থাৎ:

Promise Queue

→ often sequential

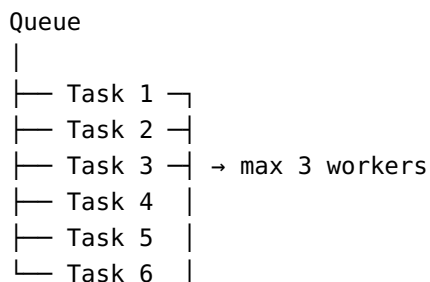
Concurrency Limiter

→ bounded parallel/concurrent execution

তবে queue-ও concurrency support করতে পারে।

48. Queue + Concurrency

Advanced queue:



এখানে:

Concurrency = 3

এটি বাস্তবে job queues, API batching, file processing ইত্যাদিতে useful ।

49. Backpressure-এর সঙ্গে Queue-এর সম্পর্ক

Producer:

1000 tasks/sec

Consumer:

100 tasks/sec

তাহলে:

Queue
100
200
300
400
500
...

Queue unlimited হলে memory বাড়তে থাকবে।

এটাই backpressure-এর সমস্যা।

Production queue-তে consider করতে হয়:

- max queue size
- rejection
- timeout
- priority
- retry
- dead-letter queue
- concurrency

50. Async Resource Management

Modern JavaScript-এ resource cleanup-এর জন্য `using` এবং `await using` syntax এসেছে।

Concept:

```
Acquire resource
  ↓
Use resource
  ↓
Automatically dispose
```

যেমন resource হতে পারে:

- file handle
- lock
- connection
- custom resource
- temporary resource

51. `using` Declaration

Conceptually:

```
{
  using resource = acquireResource();
```

```
resource.doSomething();  
}
```

block শেষ হলে resource automatically dispose করার protocol invoke হবে।

এটি traditional:

```
const resource = acquire();  
  
try {  
  resource.use();  
} finally {  
  resource.dispose();  
}
```

এর declarative form হিসেবে ভাবতে পারো।

52. Disposable Protocol

`using` resource-এর disposal protocol-এর ওপর নির্ভর করে।

Synchronous disposal:

```
Symbol.dispose
```

Asynchronous disposal:

```
Symbol.asyncDispose
```

Conceptually:

```
using  
↓  
Symbol.dispose  
  
await using  
↓  
Symbol.asyncDispose
```

53. `await using`

যদি resource cleanup asynchronous হয়:

```
{
    await using resource =
        await acquireResource();

    await resource.use();
}
```

Block শেষ হলে asynchronous disposal `await` করা হয়।

Conceptually:

```
acquire
  ↓
use
  ↓
await dispose
  ↓
continue
```

54. Why `using` Matters

Traditional:

```
const file = await openFile();

try {
    await process(file);
} finally {
    await file.close();
}
```

Potential problem:

```
Developer forgets finally
↓
resource remains open
```

`using` intent directly expresses:

```
I own this resource
↓
dispose it automatically
```

এটি resource lifecycle-এর correctness improve করতে পারে।

55. Multiple Resources

```
{
    using a = acquireA();
    using b = acquireB();

    work(a, b);
}
```

Block exit হলে cleanup reverse order-এ হওয়া গুরুত্বপূর্ণ:

```
Acquire A
Acquire B

Dispose B
Dispose A
```

এটি stack-like resource management-এর সঙ্গে মিলে যায়।

56. `using` বনাম Garbage Collection

এখানে একটি খুব important distinction:

```
Garbage Collection
→ memory reclaim
```

Resource disposal
→ external resource cleanup

GC-এর জন্য অপেক্ষা করে:

```
file.close()
database.release()
lock.release()
socket.close()
```

করা উচিত নয়।

কারণ external resources deterministic cleanup চায়।

তাই:

Memory management এবং **resource management** আলাদা **concern**।

57. Async Resource Management-এর Error Handling

ধরো:

```
{
  await using resource = await acquire();

  await work(resource);
}
```

যদি:

```
work()
```

throw করে?

তবুও resource cleanup হওয়া উচিত।

Conceptually:

```
Acquire
↓
```

```
Work
  ↓
Error
  ↓
Dispose
  ↓
Error propagates
```

এই deterministic cleanup-ই `using` -এর বড় value ।

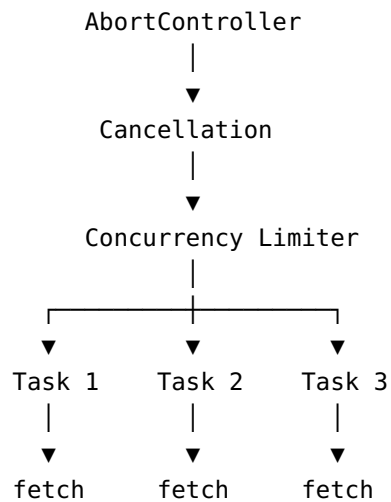
58. Abort + Concurrency একসাথে

Production system-এ patterns combine করা যায় ।

ধরো:

```
1000 API calls
max concurrency = 10
overall timeout = 10 sec
```

Architecture:



Abort হলে running requests cancel এবং pending tasks stop করা যেতে পারে ।

59. Retry + Abort

Retry loop cancellation-aware হওয়া উচিত।

খারাপ:

```
while (attempts < 5) {  
  await retry();  
}
```

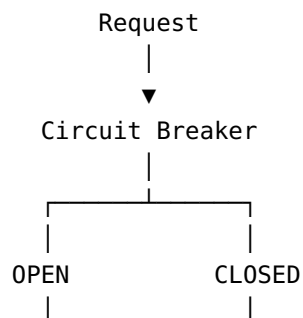
Better:

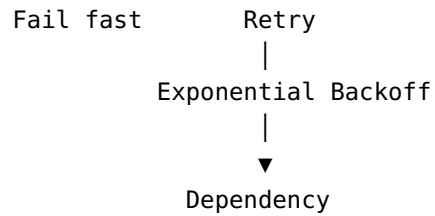
```
async function retry(fn, signal) {  
  for (let attempt = 0; attempt < 5; attempt++) {  
    signal?.throwIfAborted();  
  
    try {  
      return await fn(signal);  
    } catch (error) {  
      signal?.throwIfAborted();  
  
      await sleep(200 * 2 ** attempt);  
    }  
  }  
}
```

এতে user cancel করলে retry loop-ও বন্ধ হতে পারে।

60. Retry + Circuit Breaker

আরও advanced architecture:





Dependency continuously fail করলে:

```
failures ↑  
↓  
Circuit OPEN  
↓  
stop retries/calls
```

61. Concurrency Limiter + Retry

আরেকটি common combination:

```
1000 items  
↓  
Concurrency = 10  
↓  
Each task  
↓  
Retry max 3  
↓  
Exponential backoff
```

এতে:

```
System load bounded  
+  
Temporary failure handled
```

কিন্তু retry-এর কারণে effective request count বেড়ে যেতে পারে:

```
100 logical tasks  
+  
retries
```

=
potentially >100 physical attempts

তাই concurrency এবং retry policy একসাথে design করতে হবে।

62. Async Generator + AbortSignal

Async generator-ও cancellation-aware করা যায়।

```
async function* streamItems(signal) {  
  for (let page = 1; page <= 100; page++) {  
    signal.throwIfAborted();  
  
    const response = await fetch(  
      `/items?page=${page}`,  
      { signal }  
    );  
  
    yield response.json();  
  }  
}
```

Consumer:

```
const controller = new AbortController();  
  
try {  
  for await (  
    const items of streamItems(controller.signal)  
  ) {  
    process(items);  
  }  
} finally {  
  controller.abort();  
}
```

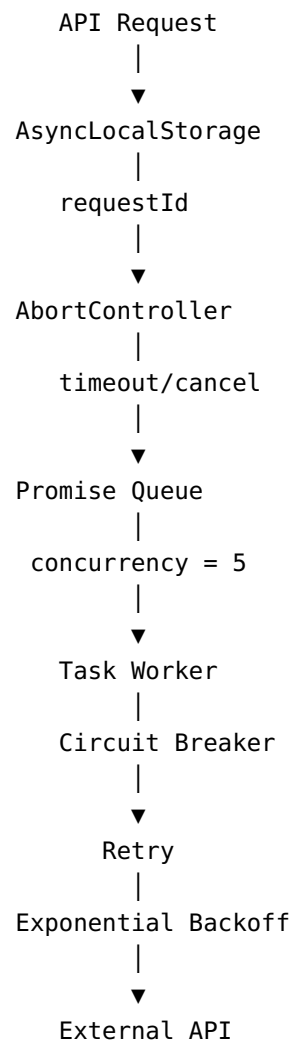
এখানে:

```
Producer  
  ↓  
Async Generator  
  ↓
```

Consumer
↓
AbortSignal
↓
Cancellation

63. Async Patterns-এর একটি Complete Architecture

একটি বড় production workflow ভাবো:



এটা বাস্তব distributed/backend system-এর খুব common ধরনের architecture ।

64. কোন Pattern কখন ব্যবহার করবে?

Problem	Pattern
Request cancel	<code>AbortController</code>
Timeout cancellation	<code>AbortSignal.timeout()</code>
Multiple cancellation sources	<code>AbortSignal.any()</code>
Async data stream	Async Generator
Async iteration	<code>for await...of</code>
Request-scoped context	<code>AsyncLocalStorage</code>
Max N concurrent jobs	Concurrency Limiter
Temporary failure	Retry
Avoid retry storm	Exponential Backoff + Jitter
Dependency repeatedly failing	Circuit Breaker
Ordered async processing	Promise Queue
Controlled async jobs	Queue + concurrency
Automatic resource cleanup	<code>using</code>
Async resource cleanup	<code>await using</code>

65. Common Mistakes

Mistake 1 — `AbortController` মানেই সব কাজ instantly stop

না।

Cancellation সাধারণত cooperative।

Mistake 2 — সব error retry করা

না।

Permanent errors retry করলে শুধু load বাড়ে।

Mistake 3 — Retry without backoff

```
fail → retry → fail → retry → fail
```

এটি retry storm তৈরি করতে পারে।

Mistake 4 — Unlimited Promise.all()

```
await Promise.all(  
  hugeArray.map(process)  
);
```

Huge input হলে resource pressure তৈরি হতে পারে।

Bounded concurrency বিবেচনা করো।

Mistake 5 — Global request context

```
let currentUser;
```

Concurrent async requests-এর জন্য unsafe।

Request-scoped context mechanism ব্যবহার করো।

Mistake 6 — Queue without limit

Producer consumer-এর চেয়ে দ্রুত হলে queue endlessly grow করতে পারে।

Mistake 7 — Circuit breaker ছাড়া aggressive retry

Dependency down হলে তোমার system dependency-কে আরও চাপ দিতে পারে।

Mistake 8 — GC-এর ওপর resource cleanup ছেড়ে দেওয়া

Memory reclaim আর file/socket/connection cleanup এক জিনিস নয়।

66. Senior-Level Design Checklist

কোনো async workflow design করার সময় নিজে থেকে প্রশ্ন করো:

Cancellation

Can this operation be cancelled?
What happens if user leaves?
What happens on timeout?

Concurrency

How many operations can run simultaneously?
Can the dependency handle that?

Failure

Which errors are retryable?
How many retries?
What backoff?
Is jitter needed?

Resilience

What happens if dependency stays down?
Do we need a circuit breaker?

Context

How do we preserve requestId/traceId?

Resources

What must be cleaned up?
Who owns it?
When is it disposed?

Backpressure

Can producer outrun consumer?
What is the maximum queue size?

67. Interview Questions

Abort

1. What is `AbortController` ?
2. What is the difference between `AbortController` and `AbortSignal` ?
3. How does fetch cancellation work?
4. How would you make your own async function cancellation-aware?
5. `AbortSignal.timeout()` কী?
6. `AbortSignal.any()` কেন useful?

Async Iteration

1. Async generator কী?
2. Generator এবং async generator-এর difference কী?
3. `for await...of` কীভাবে কাজ করে?
4. `Symbol.asyncIterator` কী?
5. Async generator কখন normal Promise-এর চেয়ে better abstraction?

Node.js Context

1. `AsyncLocalStorage` কেন দরকার?
2. Global variable দিয়ে request context রাখা dangerous কেন?
3. `AsyncLocalStorage.run()` এবং `getStore()` কী করে?
4. Logging/tracing-এ `AsyncLocalStorage` কীভাবে ব্যবহার করবে?

Concurrency

1. Concurrency limiter কেন দরকার?
2. `Promise.all()` unlimited concurrency তৈরি করতে পারে কীভাবে?
3. Concurrency এবং parallelism-এর difference কী?
4. Promise Queue এবং Concurrency Limiter-এর difference কী?
5. Backpressure কী?

Reliability

1. Retry কখন appropriate?
2. Exponential backoff কেন দরকার?
3. Jitter কেন দরকার?
4. সব HTTP error কি retry করা উচিত?

5. Retry এবং idempotency-এর relationship কী?
6. Circuit breaker-এর CLOSED/OPEN/HALF-OPEN states explain করো।
7. Circuit breaker এবং retry একসাথে কীভাবে কাজ করতে পারে?

Resource Management

1. `using` declaration কী?
 2. `await using` কখন দরকার?
 3. `Symbol.dispose` এবং `Symbol.asyncDispose` কী?
 4. Resource disposal এবং garbage collection-এর পার্থক্য কী?
-

68. Practice Tasks

Task 1 — Cancellation

একটি function বানাও:

```
fetchWithTimeout(url, timeout)
```

যেটি timeout হলে request abort করবে।

Task 2 — Async Generator

একটি async generator বানাও:

```
fetchPages()
```

যেটি:

```
page 1  
page 2  
page 3  
...
```

একেকটি page yield করবে।

Task 3 — Concurrency Limiter

Implement:

```
mapWithConcurrency(  
  items,  
  5,  
  worker  
);
```

Requirement:

```
100 items  
max 5 active
```

Task 4 — Retry

Implement:

```
retry(fn, {  
  retries: 5,  
  baseDelay: 200  
});
```

যেখানে:

```
200ms  
400ms  
800ms  
1600ms  
...
```

backoff হবে।

তারপর jitter যোগ করো।

Task 5 — Circuit Breaker

States:

```
CLOSED
OPEN
HALF_OPEN
```

সহ implement করো।

Task 6 — Promise Queue

একটি queue বানাও:

```
queue.add(task)
```

এবং ensure করো:

```
Task A
Task B
Task C
```

orderedভাবে execute হয়।

Task 7 — Combine Everything

একটি fake API processor বানাও:

```
1000 items
max concurrency = 10
retry = 3
exponential backoff
AbortSignal
circuit breaker
```

এটা implement করতে পারলে এই section-এর বেশিরভাগ production concepts practically বুঝে যাবে।

69. Quick Revision

```
AbortController
→ cancel async operation
```

AbortSignal

→ cancellation state/signal

Async Generator

→ produce async values lazily

for await...of

→ consume async iterables

AsyncLocalStorage

→ propagate request-scoped context

Concurrency Limiter

→ max K operations at once

Retry

→ recover from temporary failure

Exponential Backoff

→ progressively delay retries

Jitter

→ randomize retry timing

Circuit Breaker

→ stop hammering failing dependency

Promise Queue

→ control/order async jobs

using

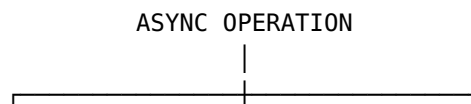
→ automatic synchronous resource disposal

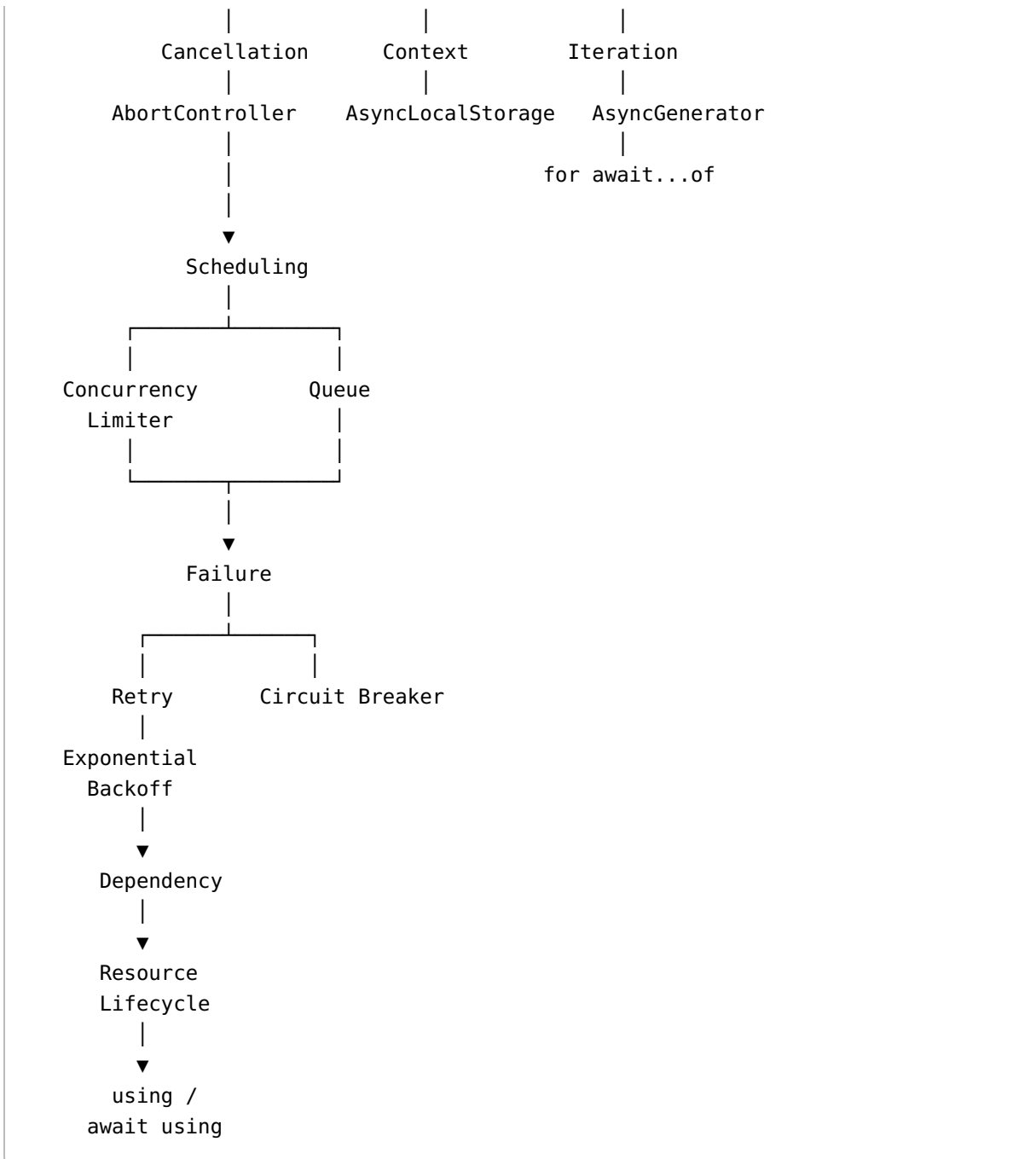
await using

→ automatic asynchronous resource disposal

70. Final Mental Model

এই পুরো Advanced Async section-টা একটি system হিসেবে ভাবো:





সবচেয়ে গুরুত্বপূর্ণ Senior-level lesson:

Advanced async মানে শুধু **asynchronous code** লেখা না; **asynchronous work**-এর **lifecycle control** করা।

একটি production-grade async operation-এর lifecycle ideally তুমি এভাবে চিন্তা করবে:

```
Start
↓
Context
↓
Concurrency control
↓
Execute
↓
Cancellation possible
↓
Temporary failure?
├─ Yes → Retry + Backoff
└─ No
↓
Repeated failure?
├─ Yes → Circuit Breaker
└─ No
↓
Consume / Produce asynchronously
↓
Cleanup resources
↓
Finish
```

আর এই 9টা topic-এর মধ্যে সবচেয়ে বেশি practical value যোগুলোর:

```
AbortController
Concurrency Limiter
Retry + Exponential Backoff
Circuit Breaker
Promise Queue
AsyncLocalStorage
```

কারণ এগুলো জানলে তুমি শুধু `async/await` ব্যবহারকারী developer না থেকে **reliable asynchronous system design** করার দিকে যেতে পারবে।